RDFIA 2026–2027 – Master IMA

Handout: Convolutional Networks, Automatic Differentiation and Batch Normalization

Clément Rambour
Sorbonne Université – ISIR, équipe MLIA

Companion to lecture 3 (*Implementation* and *Deep architectures*)

This handout first recaps the building blocks of convolutional networks: the standard *activation functions*, then the *convolution* and *pooling* layers and how to compute the size of their outputs. It then details two parts of the lecture that go fast on the slides: how deep learning libraries compute gradients (*automatic differentiation*), how to *initialize the weights* so that signals and gradients neither explode nor vanish, and the *batch normalization* layer, whose backward pass is a good exercise in applying automatic differentiation. All code listings are in the `code/` folder next to this document and run as is with PyTorch ($\geq$ 2.0).

Contents

1 Activation functions **2**

2 Convolutional neural networks **3**
2.1 Why convolutions? . . . 3
2.2 The convolution layer . . . 3
2.3 Size of the activation maps . . . 4
2.4 Pooling layers . . . 4
2.5 A typical architecture . . . 5

3 Automatic differentiation **7**
3.1 The problem . . . 7
3.2 Computation graphs . . . 7
3.3 Forward mode and reverse mode . . . 8
3.4 Never build the Jacobian: VJPs on tensors . . . 8

4 Weight initialization **10**
4.1 Why initialization matters . . . 10
4.2 Variance of a layer . . . 10
4.3 Xavier (Glorot) initialization . . . 11
4.4 Kaiming (He) initialization . . . 11
4.5 In practice . . . 11

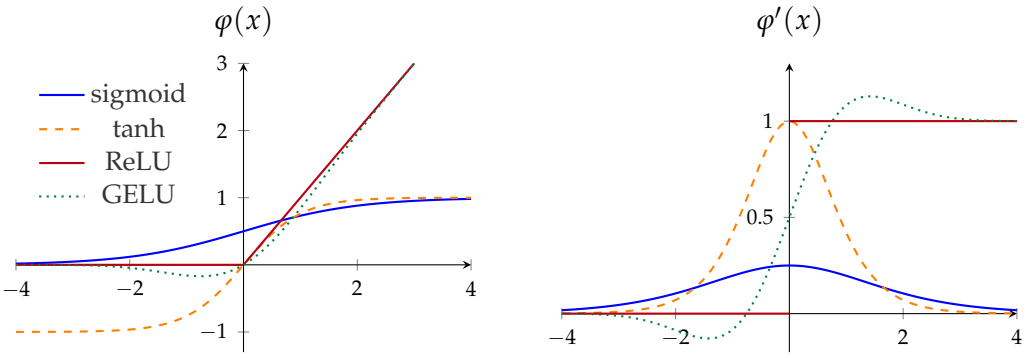
5 Batch normalization **13**
5.1 Motivation . . . 13
5.2 Forward pass . . . 13
5.3 Backward pass . . . 13
5.4 Training versus inference . . . 15
5.5 Limitations and alternatives . . . 15

1 Activation functions

Why a non-linearity? A layer computes an affine map followed by an element-wise function φ, the *activation*: $\mathbf{h} = \varphi(\mathbf{W}\mathbf{x} + \mathbf{b})$. Without φ, a stack of layers would collapse into a single affine map, since $\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)$: depth would bring nothing. With a non-linear φ, a network with one hidden layer is already a universal approximator, and depth makes the approximation far more efficient.

Standard activations. During training, the gradient reaching a layer is multiplied by φ' at every layer it goes through (Section 3.4): the shape of φ' matters as much as the shape of φ.

Name	$\varphi(x)$	$\varphi'(x)$	Range
Sigmoid	$\sigma(x) = \dfrac{1}{1+e^{-x}}$	$\sigma(x)\,(1-\sigma(x)) \leq \frac{1}{4}$	$(0,1)$
Hyperbolic tangent	$\tanh(x) = 2\sigma(2x) - 1$	$1 - \tanh^2(x) \leq 1$	$(-1,1)$
ReLU	$\max(0,x)$	$\mathbb{1}[x > 0]$	$[0,+\infty)$
Leaky ReLU	$\max(\alpha x, x)$, $\alpha = 0.01$	1 if $x > 0$, α otherwise	$\mathbb{R}$
ELU	x if $x > 0$, $\alpha(e^x - 1)$ otherwise	1 if $x > 0$, αe^x otherwise	$(-\alpha, +\infty)$
GELU	$x\,\Phi(x)$ (Φ: Gaussian c.d.f.)	$\Phi(x) + x\,\phi(x)$	$\approx (-0.17, +\infty)$
SiLU / Swish	$x\,\sigma(x)$	$\sigma(x)\,(1 + x(1-\sigma(x)))$	$\approx (-0.28, +\infty)$



Saturating activations. The sigmoid and tanh are flat for large $|x|$: their derivative tends to 0 (they *saturate*). Since $\sigma' \leq 1/4$, the gradient through L sigmoid layers is multiplied by a factor at most 4^{-L} (times the weights): this is the *vanishing gradient* problem, which made deep networks hard to train before ReLU. The sigmoid is also not zero-centered (its outputs are all positive), which slows down gradient descent; tanh is zero-centered and usually preferred to the sigmoid as a hidden activation.

ReLU and its variants. The *rectified linear unit* does not saturate for $x > 0$ (derivative 1), is very cheap, and gives sparse activations. Its drawback is *dead neurons*: a unit whose input is always negative has a zero gradient and never recovers (*e.g.* after a too large learning rate step). Leaky ReLU and ELU keep a small gradient for $x < 0$. GELU and SiLU are smooth versions of ReLU, standard in Transformers (GELU is used in the ViT of the lab). At $x = 0$, ReLU is not differentiable: libraries simply use $\varphi'(0) = 0$.

Output layer. The last layer uses an activation adapted to the task, not to optimization: none (raw scores) for regression; the sigmoid for binary or multi-label classification, $p = \sigma(s)$; the *softmax* for

classification into K classes,

$$\text{softmax}(\mathbf{s})_k = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_j}},$$

which turns the scores (*logits*) $\mathbf{s} \in \mathbb{R}^K$ into a probability distribution. It is always combined with the cross-entropy loss, and in PyTorch it is *included* in the loss (`nn.CrossEntropyLoss` takes the logits, see Section 3.4).

In practice. ReLU in the hidden layers of CNNs, GELU in Transformers; no sigmoid or tanh in deep hidden layers; softmax (inside the loss) for classification. The initialization must match the activation: He initialization for ReLU, Xavier/Glorot for tanh (Section 4).

2 Convolutional neural networks

2.1 Why convolutions?

A fully-connected layer on images does not scale: for a 1000×1000 image and 10^6 hidden units, it would have 10^{12} weights. It also ignores the structure of images: it must learn separately to detect a pattern at every position. Convolutional layers rely on two assumptions that are true for images:

- **Locality (sparse connectivity):** each output only depends on a small neighbourhood of the input (a $k \times k$ window).
- **Stationarity (weight sharing):** the same filter is applied at every position, so a pattern is detected wherever it appears. The layer is *translation equivariant*: translating the input translates the output.

The number of weights no longer depends on the image size, and the same layer can process images of any size.

2.2 The convolution layer

Input and output. A convolution layer takes a tensor of C_{in} *feature maps* (or channels) of size $H \times W$ (for the first layer, the $C_{\text{in}} = 3$ RGB channels), and outputs C_{out} feature maps, also called *activation maps*. It has C_{out} filters, each of size $C_{\text{in}} \times k \times k$: a filter always covers *all* the input channels. Each filter produces one output map:

$$Y_{o,i,j} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} W_{o,c,u,v}\ \tilde{X}_{c,\, s\,i+u,\, s\,j+v},$$

where $\tilde{X}$ is the input padded with zeros (see below), s the stride, and i, j the output positions, starting at 0. This is a cross-correlation (the kernel is not flipped), but deep learning libraries call it a convolution: since the filters are learned, the difference does not matter.

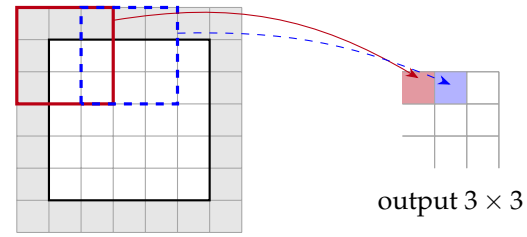
Number of parameters.

$$\boxed{\#\text{params} = C_{\text{out}} \times (C_{\text{in}} \times k \times k + 1)}$$

(the $+1$ is the bias of each filter), independently of H and W. Each output value costs $C_{\text{in}}k^2$ multiply-adds, so the layer costs $C_{\text{out}}H_{\text{out}}W_{\text{out}}C_{\text{in}}k^2$ multiply-adds: convolutions hold few parameters but most of the computation. A 1×1 convolution ($k = 1$) mixes the channels at each position without looking at the neighbours: it is a fully-connected layer applied to each pixel.

Stride. The stride s is the step between two consecutive positions of the filter. With $s = 1$ the filter visits every position; with $s = 2$ it skips one position out of two, which divides the output size by about 2 (a sub-sampling of the output).

Padding. Without padding, the filter must fit inside the input, so the output is smaller than the input ($k-1$ positions are lost along each dimension) and the borders are seen less often than the centre. Padding adds p rows and columns of zeros on each side of the input before the convolution. It is used to control the output size, in particular to keep it equal to the input size, so that the spatial size is only reduced by pooling layers (or strided convolutions).



input 5×5 (white)
padding $p = 1$ (gray)

output 3×3

kernel $k = 3$, stride $s = 2$:
two consecutive positions of the filter (red, then blue dashed),
$\lfloor (5+2-3)/2 \rfloor + 1 = 3$.

2.3 Size of the activation maps

Along each spatial dimension, the padded input has $H + 2p$ positions; the first filter position starts at 0 and the last one must end before $H + 2p$, with steps of s. The number of positions is therefore

$$\boxed{H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \qquad W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1}$$

and the output tensor has size $C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}$: its depth is the number of filters, not the number of input channels. When s does not divide $H + 2p - k$, the last rows or columns of the input are simply not visited (hence the floor). In PyTorch (`nn.Conv2d(C_in, C_out, kernel_size=k, stride=s, padding=p)`), the general formula also has a dilation d (filter with holes, default 1): $H_{\text{out}} = \lfloor (H + 2p - d(k-1) - 1)/s \rfloor + 1$.

Examples.

- 28×28 input, $k = 5$, $p = 0$, $s = 1$: $\lfloor 23/1 \rfloor + 1 = 24$, a 24×24 map.
- 32×32 input, $k = 3$, $p = 1$, $s = 2$: $\lfloor 31/2 \rfloor + 1 = 16$, a 16×16 map.
- 224×224 input, $k = 7$, $p = 3$, $s = 2$ (first layer of ResNet): $\lfloor 223/2 \rfloor + 1 = 112$.

2.4 Pooling layers

A pooling layer *spatially downsamples* each feature map independently: a window of size $k \times k$ slides over the map with stride s (and possibly padding p), and each window is summarized by a single value, its **maximum** (max pooling) or its **mean** (average pooling). The output size follows the *same formula* as the convolution, and the number of channels is unchanged. The most common setting is $k = s = 2$, $p = 0$, which divides the height and width by 2:

$$\begin{pmatrix} 1 & 3 & 2 & 1 \\ 4 & 6 & 5 & 0 \\ 3 & 1 & 1 & 2 \\ 0 & 2 & 4 & 3 \end{pmatrix} \xrightarrow{\text{max, } k=s=2} \begin{pmatrix} 6 & 5 \\ 3 & 4 \end{pmatrix}, \qquad \xrightarrow{\text{average, } k=s=2} \begin{pmatrix} 3.5 & 2 \\ 1.5 & 2.5 \end{pmatrix}.$$

Pooling has **no parameter** to learn. It reduces the computation of the next layers, quickly enlarges the region of the image seen by the following neurons, and makes the representation robust to small translations. The backward pass of max pooling sends the gradient only to the position of the maximum of each window. In PyTorch, `nn.MaxPool2d(k, stride=s, padding=p)` rounds the output size down by default; with `ceil_mode=True` it rounds it up (the last window is then partial). A *global average pooling* averages each map over all its positions, turning a $C \times H \times W$ tensor into a vector of size C, whatever H and W.

2.5 A typical architecture

A classical CNN alternates convolutions and non-linearities, regularly followed by a pooling layer, then ends with one or a few fully-connected layers:

$$\big[\,(\text{Conv} \to \text{ReLU}) \times n \to \text{Pool}\,\big] \times m$$
$$\to \ \text{Flatten (or global pooling)} \ \to \ \text{FC} \to \text{ReLU} \to \text{FC} \to \text{softmax}.$$

Along the network, the spatial size decreases while the number of channels increases: the first layers detect local, low-level patterns (edges, colours), and deeper layers, whose neurons depend on larger regions of the image, detect more abstract structures (textures, object parts, objects). The convolutional part is a *feature extractor*; the fully-connected part is a *classifier*.

Worked example: LeNet-5 on MNIST ($1 \times 28 \times 28$ inputs, tanh activations, which do not change the sizes):

Layer	Hyper-parameters	Output size	Parameters
conv1	6 filters 5×5, $p = 2$, $s = 1$	$6 \times 28 \times 28$	$6 \times (1 \cdot 25 + 1) = 156$
pool1	max 2×2, $s = 2$	$6 \times 14 \times 14$	0
conv2	16 filters 5×5, $p = 0$, $s = 1$	$16 \times 10 \times 10$	$16 \times (6 \cdot 25 + 1) = 2416$
pool2	max 2×2, $s = 2$	$16 \times 5 \times 5 = 400$	0
fc3	fully-connected	120	$400 \cdot 120 + 120 = 48\,120$
fc4	fully-connected	84	$120 \cdot 84 + 84 = 10\,164$
fc5	fully-connected	10	$84 \cdot 10 + 10 = 850$
Total			61 706

As is typical, most parameters are in the first fully-connected layer, while most of the computation is in the convolutions. Later architectures follow the same template: AlexNet (ReLU, dropout, GPU training), VGG (only 3×3 convolutions: two stacked 3×3 layers see a 5×5 region with $2 \cdot 9 = 18$ weights per channel pair instead of 25, and an extra non-linearity), ResNet (residual connections, batch normalization, global average pooling instead of large fully-connected layers).

In PyTorch, LeNet-5 is written as a `nn.Module` with the feature extractor and the classifier as two `nn.Sequential` blocks; the comments give the output size of each layer, as in the table above. The last layer returns the logits: the softmax is included in `nn.CrossEntropyLoss`.

```
import torch
import torch.nn as nn

class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(                # feature extractor
            nn.Conv2d(1, 6, kernel_size=5, padding=2),  # 6 x 28 x 28
            nn.Tanh(),
            nn.MaxPool2d(kernel_size=2, stride=2),      # 6 x 14 x 14
            nn.Conv2d(6, 16, kernel_size=5),            # 16 x 10 x 10
            nn.Tanh(),
            nn.MaxPool2d(kernel_size=2, stride=2),      # 16 x 5 x 5
        )
        self.classifier = nn.Sequential(              # classifier
            nn.Flatten(),                               # 400
            nn.Linear(16 * 5 * 5, 120),
```

```
            nn.Tanh(),
            nn.Linear(120, 84),
            nn.Tanh(),
            nn.Linear(84, num_classes),                # logits: softmax is in the loss
        )

    def forward(self, x):
        return self.classifier(self.features(x))

model = LeNet5()
x = torch.randn(8, 1, 28, 28)                          # a batch of 8 MNIST images
print(model(x).shape)                                  # torch.Size([8, 10])
print(sum(p.numel() for p in model.parameters()))      # 61706
```

3 Automatic differentiation

3.1 The problem

Training a network means running gradient descent on a scalar loss $\mathcal{L}(\mathbf{w})$ with respect to millions (or billions) of parameters:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \, \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}^t).$$

We need the *numerical value* of $\nabla_{\mathbf{w}} \mathcal{L}$ at the current point, exactly and cheaply. There are three ways to get it.

Numerical differentiation. Finite differences,

$$\frac{\partial \mathcal{L}}{\partial w_j} \approx \frac{\mathcal{L}(\mathbf{w} + h\,\mathbf{e}_j) - \mathcal{L}(\mathbf{w} - h\,\mathbf{e}_j)}{2h},$$

are easy to implement but (i) approximate: truncation error in $O(h^2)$, plus rounding error in that explodes for small h; and (ii) far too expensive: one or two evaluations of $\mathcal{L}$ *per parameter*. They remain useful to *check* a gradient implementation (Section **??**).

Symbolic differentiation. A computer algebra system applies the rules of calculus to the formula of $\mathcal{L}$ and returns a formula for the derivative. The result is exact, but expressions blow up: the derivative of a composition of L functions contains repeated sub-expressions, and without sharing them the size of the expression can grow exponentially with L (*expression swell*).

Automatic differentiation (AD). AD applies symbolic differentiation only at the level of *elementary operations* (addition, product, exp, log, matrix product, convolution, ...) whose derivatives are known, and combines them *numerically* with the chain rule, storing intermediate values instead of formulas. The result is exact up to floating-point rounding, and, in reverse mode, costs only a small constant factor more than evaluating $\mathcal{L}$ itself. Back-propagation is reverse-mode AD applied to neural networks.

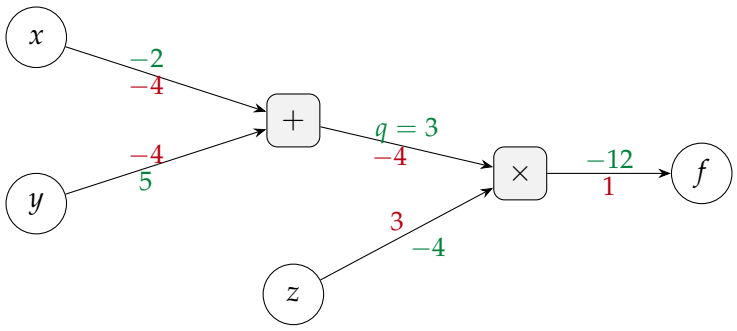
3.2 Computation graphs

Any program that computes $\mathcal{L}$ can be unrolled into a sequence of elementary operations

$$v_k = \varphi_k\big(v_j\big)_{j \in \mathrm{pa}(k)}, \qquad k = 1, \ldots, K,$$

where $\mathrm{pa}(k)$ are the *parents* of v_k (the inputs of the operation). The inputs and parameters are the first nodes, and the output is the last one. This defines a directed acyclic graph: the *computation graph*.

Running example (from the slides). $f(x, y, z) = (x + y)\, z$ at $x = -2$, $y = 5$, $z = -4$. The intermediate variable is $q = x + y$.



Green: values (forward pass). Red: adjoints $\partial f / \partial v$ (backward pass).

The *forward pass* evaluates the nodes in topological order: $q = 3$, $f = -12$. The *backward pass* starts from $\partial f/\partial f = 1$ and goes back through each operation using its local derivative:

$$f = q\,z \;\Rightarrow\; \frac{\partial f}{\partial q} = z = -4, \qquad \frac{\partial f}{\partial z} = q = 3,$$
$$q = x + y \;\Rightarrow\; \frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\,\frac{\partial q}{\partial x} = -4 \times 1 = -4, \qquad \frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\,\frac{\partial q}{\partial y} = -4.$$

Note that the backward pass needs the values computed in the forward pass (q and z for the product). This is why the forward values must be stored.

3.3 Forward mode and reverse mode

Consider a function $F = F_L \circ \cdots \circ F_1 : \mathbb{R}^n \to \mathbb{R}^m$. By the chain rule, its Jacobian is a product of the Jacobians J_ℓ of the elementary steps:

$$J_F = J_L\, J_{L-1} \cdots J_1 \in \mathbb{R}^{m \times n}.$$

This product can be evaluated in two orders.

Forward mode (tangents). Choose an input direction $\dot{\mathbf{x}} \in \mathbb{R}^n$ and propagate it *with* the computation, from the inputs to the outputs: $\dot{v}_k = \sum_{j \in \mathrm{pa}(k)} \frac{\partial \varphi_k}{\partial v_j}\,\dot{v}_j$. One pass gives the *Jacobian-vector product* (JVP) $J_F\,\dot{\mathbf{x}} = J_L(\cdots(J_1\dot{\mathbf{x}}))$, *i.e.* one column of J_F when $\dot{\mathbf{x}} = \mathbf{e}_j$. The full Jacobian requires n passes.

Reverse mode (adjoints). Choose an output direction $\bar{\mathbf{y}} \in \mathbb{R}^m$ and propagate it *backwards*, from the outputs to the inputs. Each node accumulates the contributions of all its children:

$$\bar{v}_j = \sum_{k\,:\,j \in \mathrm{pa}(k)} \bar{v}_k\,\frac{\partial \varphi_k}{\partial v_j}, \qquad \text{with } \bar{v}_j = \frac{\partial(\bar{\mathbf{y}}^\top F)}{\partial v_j}.$$

One pass gives the *vector-Jacobian product* (VJP) $\bar{\mathbf{y}}^\top J_F = ((\bar{\mathbf{y}}^\top J_L)J_{L-1}) \cdots J_1$, *i.e.* one row of J_F. The full Jacobian requires m passes.

Why deep learning uses reverse mode. The loss is a scalar: $m = 1$ and n = number of parameters is huge. Reverse mode returns the whole gradient $\nabla_{\mathbf{w}}\mathcal{L}$ in *one* forward and *one* backward pass; forward mode would need n passes. The cost of the backward pass is a small multiple of the forward pass: typically 2–3$\times$ in practice, and bounded by a small constant (about 4–5) in operation count [5].

The price of reverse mode is *memory*: all intermediate values (activations) needed by the local derivatives must be kept until the backward pass reaches them. For a deep network the activations, not the weights, usually dominate GPU memory. *Gradient checkpointing* [3] trades memory for compute: it stores only some activations and recomputes the others during the backward pass (`torch.utils.checkpoint`).

On the running example, forward mode from x ($\dot{x} = 1$, $\dot{y} = \dot{z} = 0$) gives $\dot{q} = 1$, $\dot{f} = \dot{q}\,z + q\,\dot{z} = -4 = \partial f/\partial x$: one derivative per pass, while the single backward pass above gave all three.

3.4 Never build the Jacobian: VJPs on tensors

In a network, the nodes are tensors and the local Jacobians are gigantic. Each operation therefore implements its VJP directly, without building its Jacobian: given the adjoint of its output, it returns the adjoint of its inputs.

Linear layer. Take a batch $\mathbf{X} \in \mathbb{R}^{N\times m}$, $\mathbf{W} \in \mathbb{R}^{m\times K}$, $\mathbf{b} \in \mathbb{R}^K$ and scores $\mathbf{S} = \mathbf{X}\mathbf{W} + \mathbf{1}\mathbf{b}^\top \in \mathbb{R}^{N\times K}$. With the values of the slides ($N = 100$, $m = 4096$, $K = 1000$), the Jacobian $\partial\mathbf{S}/\partial\mathbf{W}$ has $(NK) \times (mK) \approx 4 \cdot 10^{11}$ entries, 3.2 TB in double precision. Yet, writing $\Delta = \partial\mathcal{L}/\partial\mathbf{S}$ (same shape as $\mathbf{S}$, only 10^5 entries), the chain rule

$$\frac{\partial \mathcal{L}}{\partial W_{jk}} = \sum_i \Delta_{ik} \frac{\partial S_{ik}}{\partial W_{jk}} = \sum_i \Delta_{ik}\, X_{ij}$$

gives directly

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{X}^\top \Delta}, \qquad \boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \Delta\, \mathbf{W}^\top}, \qquad \boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \Delta^\top \mathbf{1}}.$$

Two matrix products and a sum, with the same cost as the forward pass. The bias gradient illustrates a general rule: *the VJP of a broadcast is a sum* over the broadcast axis.

Softmax + cross-entropy. For one example with scores $\mathbf{s} \in \mathbb{R}^K$ and true class c, the loss is

$$\ell(\mathbf{s}) = -\log \mathrm{softmax}(\mathbf{s})_c = -s_c + \log \sum_k e^{s_k},$$

hence

$$\frac{\partial \ell}{\partial s_k} = \mathrm{softmax}(\mathbf{s})_k - \mathbb{1}[k = c], \qquad \textit{i.e.} \qquad \Delta = \tfrac{1}{N}(\hat{\mathbf{Y}} - \mathbf{Y}^*)$$

for the mean over a batch. Libraries fuse softmax and cross-entropy into a single operation, which is why `F.cross_entropy` takes raw scores (*logits*) as input: the fused form is both simpler and numerically stable. The log-sum-exp is computed as

$$\log \sum_k e^{s_k} = M + \log \sum_k e^{s_k - M}, \qquad M = \max_k s_k,$$

which neither overflows nor takes the log of an underflowed zero.

The listing below checks these formulas against PyTorch's autograd.

```
import torch

N, m, K = 100, 4096, 1000
X = torch.randn(N, m)
W = torch.randn(m, K, requires_grad=True)
b = torch.zeros(K, requires_grad=True)
labels = torch.randint(0, K, (N,))

S = X @ W + b
loss = torch.nn.functional.cross_entropy(S, labels)
loss.backward()

# Manual backward: never builds the (N*K) x (m*K) Jacobian dS/dW
Y_hat = torch.softmax(S, dim=1).detach()
Y_star = torch.nn.functional.one_hot(labels, K).float()
Delta = (Y_hat - Y_star) / N        # dL/dS, shape N x K
dW = X.T @ Delta                    # m x K
db = Delta.sum(0)                   # K

print(torch.allclose(dW, W.grad, atol=1e-6), torch.allclose(db, b.grad, atol=1e-6))
```

4 Weight initialization

4.1 Why initialization matters

Training starts from random weights, and their *scale* matters a lot. In the forward pass, the signal goes through every layer; in the backward pass, the gradient goes through every layer again, in reverse (Section 3.4). If each layer multiplies the variance of the signal by a factor λ, after L layers it is multiplied by λ^L: with $\lambda = 1.5$ and $L = 30$, by $1.5^{30} \approx 2 \cdot 10^5$ (*exploding* signals and gradients); with $\lambda = 0.5$, by 10^{-9} (*vanishing* ones, the network does not learn). A good initialization keeps the variance of the activations, and of the gradients, roughly constant across the layers: $\lambda = 1$.

Why random? If all the weights of a layer start equal (*e.g.* all zero), all its neurons compute the same output and receive the same gradient: they stay identical forever. Random weights *break the symmetry*. Biases, in contrast, are usually initialized to 0.

4.2 Variance of a layer

Consider a layer $\mathbf{y} = \mathbf{W}\mathbf{x}$ with n inputs and n' outputs (biases at 0), so that

$$y_i = \sum_{j=1}^{n} W_{ij}\, x_j.$$

We assume that the weights W_{ij} are independent, identically distributed, with zero mean, and independent of the inputs, and that the inputs x_j are identically distributed. Then, for each i,

$$\begin{aligned}
\mathrm{Var}[y_i] &= \sum_{j=1}^{n} \mathrm{Var}[W_{ij}\, x_j] && \text{(independence of the terms)} \\
&= \sum_{j=1}^{n} \mathrm{Var}[W_{ij}]\ \mathbb{E}[x_j^2] && \text{(independence of } W_{ij} \text{ and } x_j\text{, and } \mathbb{E}[W_{ij}] = 0\text{)} \\
&= n\,\mathrm{Var}[w]\ \mathbb{E}[x^2] && \text{(identical distributions).}
\end{aligned}$$

The middle step uses $\mathrm{Var}[wx] = \mathbb{E}[w^2]\,\mathbb{E}[x^2] - \mathbb{E}[w]^2\,\mathbb{E}[x]^2 = \mathrm{Var}[w]\,\mathbb{E}[x^2]$ when $\mathbb{E}[w] = 0$: the input does not need to be zero-mean.

Linear activation. If the inputs are zero-mean, $\mathbb{E}[x^2] = \mathrm{Var}[x]$ and

$$\mathrm{Var}[y] = n\,\mathrm{Var}[w]\,\mathrm{Var}[x], \qquad \text{and after } L \text{ layers} \qquad \mathrm{Var}[y_L] = \Big(\prod_{\ell=1}^{L} n_\ell\,\mathrm{Var}[w_\ell]\Big)\mathrm{Var}[x].$$

Backward pass. The gradient with respect to the input is $\bar{\mathbf{x}} = \mathbf{W}^\top \bar{\mathbf{y}}$ (Section 3.4): each $\bar{x}_j = \sum_i W_{ij}\,\bar{y}_i$ sums n' terms, so the same computation gives

$$\mathrm{Var}\Big[\frac{\partial \mathcal{L}}{\partial \mathbf{x}}\Big] = \Big(\prod_{\ell=1}^{L} n'_\ell\,\mathrm{Var}[w_\ell]\Big)\mathrm{Var}\Big[\frac{\partial \mathcal{L}}{\partial \mathbf{y}_L}\Big].$$

The signal explodes if $n_\ell\,\mathrm{Var}[w_\ell] > 1$ and vanishes if it is < 1; the gradient does the same with n'_ℓ.

Fan-in and fan-out in a CNN. n and n' are the number of connections feeding into and out of a neuron:

$$n = \text{fan_in} = C_{\text{in}} \times k_h \times k_w, \qquad n' = \text{fan_out} = C_{\text{out}} \times k_h \times k_w,$$

with $k_h \times k_w$ the kernel size. A fully-connected layer can be viewed as a convolution whose kernel covers its whole input: on a $C \times H \times W$ input, $n = C \cdot H \cdot W$ is its number of inputs and n' its number of outputs.

4.3 Xavier (Glorot) initialization

For a linear activation, keeping the variance constant requires $n \operatorname{Var}[w] = 1$ in the forward pass and $n' \operatorname{Var}[w] = 1$ in the backward pass. Both cannot hold when $n \neq n'$; Glorot and Bengio [4] take the compromise

$$\boxed{\operatorname{Var}[w] = \frac{2}{n + n'}}$$

(*i.e.* replace n by the average $(n + n')/2$). The weights are drawn either from a Gaussian $\mathcal{N}\left(0, \frac{2}{n+n'}\right)$ (`nn.init.xavier_normal_`) or from a uniform distribution $\mathcal{U}(-a, a)$, which has variance $a^2/3$, hence $a = \sqrt{6/(n + n')}$ (`nn.init.xavier_uniform_`). LeCun et al. [8] proposed the forward-only version $\operatorname{Var}[w] = 1/n$. This analysis also applies to tanh, which is close to the identity around 0 ($\tanh'(0) = 1$). With a poor initialization, Glorot and Bengio observed that the gradients of the first layers are much smaller than those of the last ones; with Xavier initialization, all layers have gradients of similar scale.

4.4 Kaiming (He) initialization

With ReLU, the input of a layer is $\mathbf{x} = \mathrm{ReLU}(\mathbf{y}_{\text{prev}})$, the output of the previous layer after the activation: it is *not* zero-mean, so we keep $\mathbb{E}[x^2]$ in the variance formula. If $\mathbf{y}_{\text{prev}}$ is zero-mean and symmetric (which holds when the previous weights are), ReLU keeps exactly the positive half of its distribution and

$$\mathbb{E}[x^2] = \mathbb{E}\left[\mathrm{ReLU}(y_{\text{prev}})^2\right] = \tfrac{1}{2}\,\mathbb{E}[y_{\text{prev}}^2] = \tfrac{1}{2}\operatorname{Var}[y_{\text{prev}}],$$

so that

$$\operatorname{Var}[y] = \tfrac{1}{2}\, n \operatorname{Var}[w] \operatorname{Var}[y_{\text{prev}}], \qquad \text{and after } L \text{ layers} \qquad \operatorname{Var}[y_L] = \Big(\prod_{\ell=1}^{L} \tfrac{1}{2}\, n_\ell \operatorname{Var}[w_\ell]\Big) \operatorname{Var}[y_0].$$

ReLU halves the variance at each layer: Xavier initialization makes the signal vanish by a factor 2 per layer. He et al. [6] compensate with

$$\boxed{\operatorname{Var}[w] = \frac{2}{n}} \qquad \textit{i.e.} \qquad w \sim \mathcal{N}\left(0, \frac{2}{n}\right) \quad \text{or} \quad w \sim \mathcal{U}(-a, a),\ a = \sqrt{6/n}$$

(`nn.init.kaiming_normal_`, `nn.init.kaiming_uniform_`). Using n (`mode="fan_in"`, default) preserves the forward signal, using n' (`mode="fan_out"`) the backward gradient, and either one is *sufficient*: if the forward condition holds, the backward factor of layer ℓ is n'_ℓ / n_ℓ, and in a CNN the product of these ratios telescopes (C_{out} of a layer is C_{in} of the next) to $C_{\text{out}}^{(L)} / C_{\text{in}}^{(1)}$, which does not grow with depth. For a leaky ReLU of slope α, the factor $\frac{1}{2}$ becomes $\frac{1+\alpha^2}{2}$. On VGG networks with ReLU, He et al. report that Kaiming initialization converges faster than Xavier with 22 layers, and is the only one of the two that trains at all with 30 layers.

4.5 In practice

In PyTorch, the functions of `nn.init` modify a tensor in place:

```
conv = nn.Conv2d(64, 128, kernel_size=3)
nn.init.kaiming_normal_(conv.weight, nonlinearity="relu")   # Var[w] = 2 / fan_in
nn.init.zeros_(conv.bias)
```

The listing below propagates standardized inputs through a 30-layer ReLU network of width $n = n' = 512$, initialized in four different ways, and prints the standard deviation of the activations along the network.

```python
import torch
import torch.nn as nn

def activation_std(init, depth=30, width=512):
    # Standard deviation of the activations after each layer of a deep ReLU MLP
    torch.manual_seed(0)
    layers = [nn.Linear(width, width) for _ in range(depth)]
    for layer in layers:
        init(layer.weight)
        nn.init.zeros_(layer.bias)
    h = torch.randn(1000, width)        # 1000 standardized inputs
    stds = []
    with torch.no_grad():
        for layer in layers:
            h = torch.relu(layer(h))
            stds.append(h.std().item())
    return stds

inits = {
    "N(0, 1)": lambda w: nn.init.normal_(w, std=1.0),
    "PyTorch default": lambda w: nn.init.kaiming_uniform_(w, a=5 ** 0.5),
    "Xavier": nn.init.xavier_normal_,
    "Kaiming": lambda w: nn.init.kaiming_normal_(w, nonlinearity="relu"),
}
for name, init in inits.items():
    stds = activation_std(init)
    print(f"{name:>16}: " + "  ".join(f"layer {l}: {stds[l - 1]:.1e}" for l in (1, 10,
    20, 30)))
```

Initialization	layer 1	layer 10	layer 20	layer 30
$\mathcal{N}(0,1)$	$1.3 \cdot 10^{1}$	$7.9 \cdot 10^{11}$	$6.5 \cdot 10^{23}$	`inf`
PyTorch default	$3.4 \cdot 10^{-1}$	$9.6 \cdot 10^{-5}$	$1.2 \cdot 10^{-8}$	$1.8 \cdot 10^{-12}$
Xavier	$5.9 \cdot 10^{-1}$	$2.3 \cdot 10^{-2}$	$5.2 \cdot 10^{-4}$	$1.6 \cdot 10^{-5}$
Kaiming	$8.3 \cdot 10^{-1}$	$7.2 \cdot 10^{-1}$	$5.3 \cdot 10^{-1}$	$5.2 \cdot 10^{-1}$

The measured evolution matches the theory: per layer, the standard deviation is multiplied by $\sqrt{n/2} = 16$ with $\mathcal{N}(0,1)$ weights (it overflows the `float32` range before layer 30), by $1/\sqrt{2}$ with Xavier (ReLU halves the variance), and stays constant with Kaiming. The default initialization of `nn.Linear` and `nn.Conv2d` is $\mathcal{U}(-1/\sqrt{n}, 1/\sqrt{n})$, *i.e.* $\mathrm{Var}[w] = 1/(3n)$: it multiplies the standard deviation by $1/\sqrt{6}$ per ReLU layer. It is fine for shallow networks or with batch normalization, but deep plain ReLU networks should be initialized explicitly.

Takeaways. Signals and gradients can explode or vanish, because their variance is multiplied by a *product* of scaling factors, one per layer. Initialization chooses $\mathrm{Var}[w]$ so that each factor is 1: $2/(n+n')$ for linear or tanh activations (Xavier), $2/n$ for ReLU (Kaiming). These derivations rely on strong assumptions (independence, symmetric distributions, a specific activation) that only hold at the start of training. Initialization is *necessary* but often *not sufficient*: keeping the variance under control *during* training is the role of normalization layers, such as batch normalization.

5 Batch normalization

5.1 Motivation

Gradient descent works best when the inputs of each layer are well conditioned (centered, of unit scale); this is why we normalize the data and use careful initializations (Xavier, He, Section 4). But the distribution of the inputs of a hidden layer depends on all the weights below it, and changes at every update. Ioffe and Szegedy [7] called this *internal covariate shift* and proposed to normalize hidden activations explicitly, inside the network, using statistics of the current mini-batch.

In practice batch normalization (BN) allows much larger learning rates, makes training far less sensitive to initialization, speeds up convergence and has a mild regularizing effect. *Why* it works is still debated: later work [9] showed that BN does not really reduce covariate shift, and attributes its benefits to a smoother loss landscape (better-behaved gradients). Keep the mechanism in mind; the "covariate shift" story is only the original intuition.

5.2 Forward pass

Consider a mini-batch of N activations of one feature (one neuron), $x_1, \dots, x_N$. BN computes

$$\mu_B = \frac{1}{N}\sum_{i=1}^{N} x_i, \qquad \sigma_B^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu_B)^2,$$
$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma\,\hat{x}_i + \beta,$$

independently for each feature, where ϵ (*e.g.* 10^{-5}) avoids divisions by zero and γ, β are *learned* parameters (one pair per feature).

Why γ and β? Forcing every activation to be standardized would restrict what the network can represent; for instance, standardized inputs keep a sigmoid in its almost-linear regime. With $\gamma = \sqrt{\sigma_B^2 + \epsilon}$ and $\beta = \mu_B$ the layer can recover the identity, so BN never reduces expressivity: it only changes the *parametrization*, and the mean and scale of each feature become two explicit parameters instead of a complicated function of all the weights below.

Convolutional layers. For a feature map of shape $N \times C \times H \times W$, statistics are computed *per channel* over $N \times H \times W$ values, so that all spatial positions are normalized in the same way (translation equivariance is preserved); $\gamma, \beta \in \mathbb{R}^C$. This is `nn.BatchNorm2d` (and `nn.BatchNorm1d` for $N \times C$ inputs).

Placement. The original recipe places BN after the linear/convolution and before the non-linearity: `Conv` $\to$ `BN` $\to$ `ReLU`. The bias of the convolution is then useless (BN subtracts the mean, and β plays its role): use `bias=False`.

5.3 Backward pass

Unlike most layers, BN couples the examples of the batch: each y_i depends on all the x_j through μ_B and σ_B^2. Let $\bar{y}_i = \partial\mathcal{L}/\partial y_i$ be the upstream gradients, and write $s = \sqrt{\sigma_B^2 + \epsilon}$. We apply reverse mode to the computation graph $x \to (\mu_B, \sigma_B^2) \to \hat{x} \to y$.

Parameters. $y_i = \gamma\hat{x}_i + \beta$ gives

$$\frac{\partial\mathcal{L}}{\partial\gamma} = \sum_{i=1}^{N} \bar{y}_i\,\hat{x}_i, \qquad \frac{\partial\mathcal{L}}{\partial\beta} = \sum_{i=1}^{N} \bar{y}_i, \qquad g_i := \frac{\partial\mathcal{L}}{\partial\hat{x}_i} = \gamma\,\bar{y}_i.$$

Variance and mean. σ_B^2 influences every $\hat{x}_i$ through s, and μ_B influences every $\hat{x}_i$ directly and σ_B^2:

$$\frac{\partial \mathcal{L}}{\partial \sigma_B^2} = \sum_i g_i \,(x_i - \mu_B) \cdot \left(-\tfrac{1}{2}\right) s^{-3} = -\frac{1}{2s^2} \sum_i g_i \hat{x}_i,$$
$$\frac{\partial \mathcal{L}}{\partial \mu_B} = \sum_i g_i \left(-\frac{1}{s}\right) + \frac{\partial \mathcal{L}}{\partial \sigma_B^2} \cdot \underbrace{\frac{-2}{N} \sum_i (x_i - \mu_B)}_{=0} = -\frac{1}{s} \sum_i g_i.$$

Inputs. Each x_i reaches the loss through three paths: $\hat{x}_i$ directly, μ_B (with $\partial \mu_B / \partial x_i = 1/N$) and σ_B^2 (with $\partial \sigma_B^2 / \partial x_i = 2(x_i - \mu_B)/N$). Summing the three contributions:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \frac{g_i}{s} + \frac{\partial \mathcal{L}}{\partial \sigma_B^2} \frac{2(x_i - \mu_B)}{N} + \frac{\partial \mathcal{L}}{\partial \mu_B} \frac{1}{N} = \frac{g_i}{s} - \frac{\hat{x}_i}{Ns} \sum_j g_j \hat{x}_j - \frac{1}{Ns} \sum_j g_j,$$

that is, in compact form,

$$\boxed{\frac{\partial \mathcal{L}}{\partial x_i} = \frac{1}{Ns} \Big(N g_i - \sum_j g_j - \hat{x}_i \sum_j g_j \, \hat{x}_j \Big), \qquad g_i = \gamma \, \bar{y}_i.}$$

Interpretation. Up to the factor $1/s$, the input gradient is g with its mean removed and its component along $\hat{\mathbf{x}}$ removed: it is the projection of g onto the orthogonal complement of $\operatorname{span}(\mathbf{1}, \hat{\mathbf{x}})$. This is expected, since the output of BN is *invariant* to a shift ($\mathbf{x} + c\mathbf{1}$) or a rescaling ($a\mathbf{x}$) of its input, so the gradient has no component in these directions. A consequence: the weights of a layer followed by BN are scale invariant, so their norm only changes the *effective learning rate* (it scales as $1/\|\mathbf{W}\|^2$), one of the reasons BN tolerates large learning rates.

The following listing implements the forward and backward passes and checks them against autograd and `nn.BatchNorm1d`.

```
import torch

def bn_forward(x, gamma, beta, eps=1e-5):
    mu = x.mean(0)
    var = x.var(0, unbiased=False)
    x_hat = (x - mu) / torch.sqrt(var + eps)
    return gamma * x_hat + beta, (x_hat, var, eps)

def bn_backward(dy, gamma, cache):
    x_hat, var, eps = cache
    N = dy.shape[0]
    dgamma = (dy * x_hat).sum(0)
    dbeta = dy.sum(0)
    dx_hat = dy * gamma
    dx = (N * dx_hat - dx_hat.sum(0) - x_hat * (dx_hat * x_hat).sum(0)) \
         / (N * torch.sqrt(var + eps))
    return dx, dgamma, dbeta

torch.manual_seed(0)
x = torch.randn(32, 8, dtype=torch.float64, requires_grad=True)
gamma = torch.rand(8, dtype=torch.float64, requires_grad=True)
beta = torch.randn(8, dtype=torch.float64, requires_grad=True)

y, cache = bn_forward(x, gamma, beta)
dy = torch.randn_like(y)            # upstream gradient dL/dy
y.backward(dy)                      # autograd reference
```

```
with torch.no_grad():                # our backward, without autograd
    dx, dgamma, dbeta = bn_backward(dy, gamma, cache)
print(torch.allclose(dx, x.grad),
      torch.allclose(dgamma, gamma.grad),
      torch.allclose(dbeta, beta.grad))

# Same forward as the library layer (training mode, fresh layer)
layer = torch.nn.BatchNorm1d(8, dtype=torch.float64)
print(torch.allclose(layer(x), bn_forward(x, 1.0, 0.0)[0]))
```

5.4 Training versus inference

At test time, the prediction for an image must not depend on the other images of the batch (and batches may contain a single image). BN therefore has two modes:

- **Training:** normalize with the statistics (μ_B, σ_B^2) of the current batch, and update *running* estimates with an exponential moving average: $\mu_{\text{run}} \leftarrow (1-\alpha)\,\mu_{\text{run}} + \alpha\,\mu_B$ (same for the variance; in PyTorch α is called `momentum`, default 0.1, and the running variance uses the unbiased estimator).
- **Inference:** normalize with the fixed $(\mu_{\text{run}}, \sigma_{\text{run}}^2)$. BN becomes a fixed affine map per feature, which can even be folded into the weights of the preceding convolution.

In PyTorch the mode is set by `model.train()` / `model.eval()`. **Forgetting `model.eval()` before evaluating is one of the most common bugs**: results then depend on the batch composition and size.

```
import torch
import torch.nn as nn

bn = nn.BatchNorm2d(16, momentum=0.1)     # per-channel statistics over (N, H, W)
x = 3 + 2 * torch.randn(64, 16, 8, 8)

bn.train()
for _ in range(100):
    y = bn(x)                             # uses batch stats, updates running ones
print(bn.running_mean.mean().item(), bn.running_var.mean().item())   # close to 3 and 4

bn.eval()
y1, y2 = bn(x[:1]), bn(x[:2])[:1]         # eval: output independent of the batch
print(torch.allclose(y1, y2))
```

5.5 Limitations and alternatives

- **Small batches.** With few examples per batch (large images, detection, segmentation, 3D), μ_B and σ_B^2 are noisy and performance drops. In multi-GPU training, each GPU sees only its own sub-batch; `SyncBatchNorm` computes statistics across GPUs.
- **Train/test discrepancy.** Batch statistics in training versus running statistics at test time; when fine-tuning with tiny batches, BN layers are often frozen (kept in eval mode).
- **Sequences and Transformers.** Batch statistics are ill-defined over variable-length sequences.

These issues motivated normalizations that do not use the batch dimension. They all share the same formula $\gamma\,(x-\mu)\,/\,\sqrt{\sigma^2+\epsilon}+\beta$ and differ only by the axes over which μ and σ^2 are computed (input of shape $N \times C \times H \times W$):

Method	Statistics over	Uses batch?	Typical use
Batch Norm [7]	N, H, W (per channel)	yes	CNNs, large batches
Layer Norm [1]	C, H, W (per example)	no	Transformers, RNNs
Instance Norm [10]	H, W (per example & channel)	no	style transfer
Group Norm [11]	H, W + groups of channels	no	small batches

Since they do not use the batch dimension, these alternatives behave identically at training and test time, so they need no running statistics.

References

[1] J. L. Ba, J. R. Kiros, G. E. Hinton. Layer normalization. *arXiv:1607.06450*, 2016.

[2] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind. Automatic differentiation in machine learning: a survey. *JMLR*, 18(153):1–43, 2018.

[3] T. Chen, B. Xu, C. Zhang, C. Guestrin. Training deep nets with sublinear memory cost. *arXiv:1604.06174*, 2016.

[4] X. Glorot, Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. *AISTATS*, 2010.

[5] A. Griewank, A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*. SIAM, 2nd ed., 2008.

[6] K. He, X. Zhang, S. Ren, J. Sun. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. *ICCV*, 2015.

[7] S. Ioffe, C. Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. *ICML*, 2015.

[8] Y. LeCun, L. Bottou, G. B. Orr, K.-R. Müller. Efficient BackProp. In *Neural Networks: Tricks of the Trade*, Springer, 1998.

[9] S. Santurkar, D. Tsipras, A. Ilyas, A. Mądry. How does batch normalization help optimization? *NeurIPS*, 2018.

[10] D. Ulyanov, A. Vedaldi, V. Lempitsky. Instance normalization: the missing ingredient for fast stylization. *arXiv:1607.08022*, 2016.

[11] Y. Wu, K. He. Group normalization. *ECCV*, 2018.

For a broader introduction to AD, see the survey [2] and the autodiff cookbook of the JAX documentation (`https://docs.jax.dev/en/latest/notebooks/autodiff_cookbook.html`).